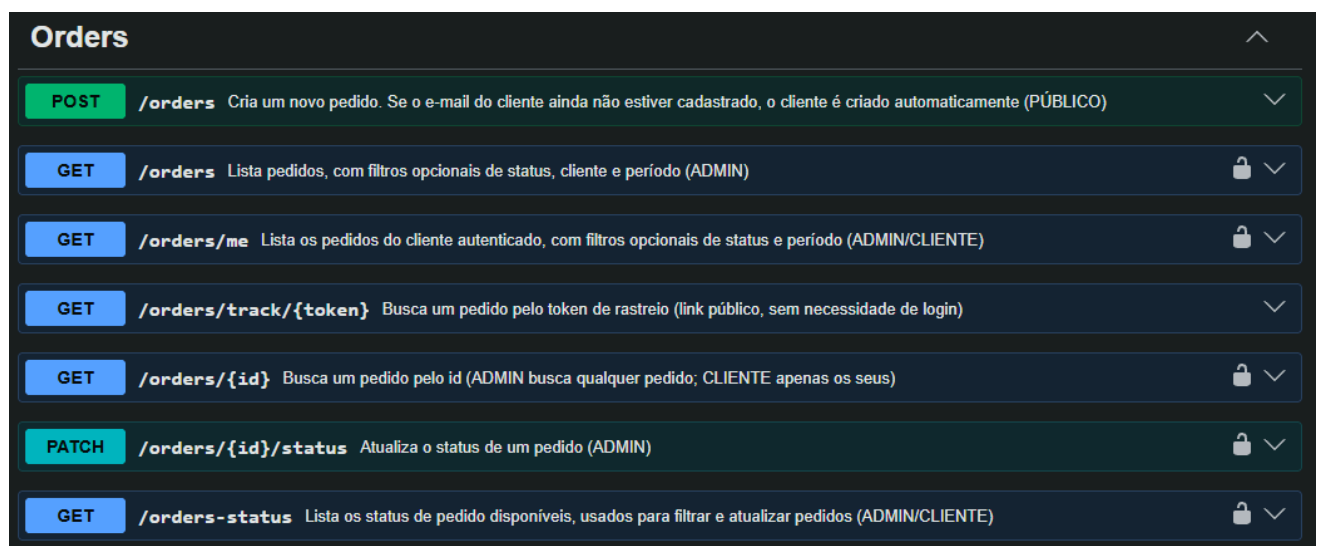


Teste Técnico – Open Finance

Foi desenvolvida uma [API](#) para controle de pedidos (delivery) utilizando nodejs com o framework Nest (<https://github.com/nestjs/nest>), TypeORM (<https://typeorm.io/>), (PostgreSQL) e Socket.IO (<https://socket.io/>), com uma interface web simples (Handlebars) para simular o fluxo de pedido de ponta a ponta.

Parte 1 – Desenvolvimento

Todos os endpoints solicitados foram implementados, principalmente os de pedidos (orders), além de endpoints extras que facilitam o uso do sistema dependendo do papel de usuário (cliente/admin). Endpoints de autenticação e para manipular produtos/clientes também foram criados.



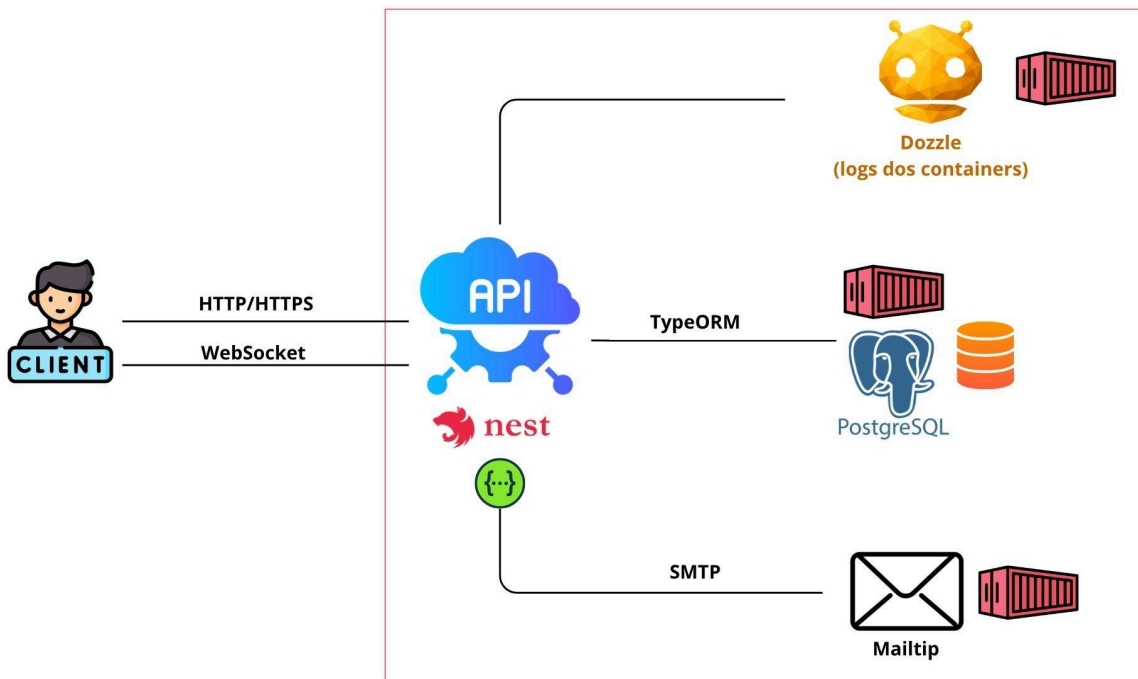
Orders		
POST	/orders	Cria um novo pedido. Se o e-mail do cliente ainda não estiver cadastrado, o cliente é criado automaticamente (PÚBLICO)
GET	/orders	Lista pedidos, com filtros opcionais de status, cliente e período (ADMIN)
GET	/orders/me	Lista os pedidos do cliente autenticado, com filtros opcionais de status e período (ADMIN/CLIENTE)
GET	/orders/track/{token}	Busca um pedido pelo token de rastreio (link público, sem necessidade de login)
GET	/orders/{id}	Busca um pedido pelo id (ADMIN busca qualquer pedido; CLIENTE apenas os seus)
PATCH	/orders/{id}/status	Atualiza o status de um pedido (ADMIN)
GET	/orders-status	Lista os status de pedido disponíveis, usados para filtrar e atualizar pedidos (ADMIN/CLIENTE)

Figura - Documentação Swagger da API, disponível em /swagger.

Parte 2 – Arquitetura

A aplicação ‘perfeita’ em produção rodaria em containers no Amazon ECS com Fargate, atrás de um Load Balancer, com várias cópias da API em diferentes zonas de disponibilidade para garantir escalabilidade e alta disponibilidade. O banco PostgreSQL ficaria no Amazon RDS (replicado), e tarefas que não precisam de resposta imediata (e-mail, estoque, analytics) seriam processadas por filas (Amazon SQS), com tentativas automáticas e uma fila de erro (Dead Letter Queue) para garantir que nenhuma mensagem seja perdida em caso de falha.

Arquitetura atual:



Parte 3 – Microsserviços e Mensageria

O envio de e-mail já está implementado no MailService (Nodemailer), disparado logo após a confirmação do pedido e mudança de status, erros são capturados e os logs salvos sem derrubar a criação do pedido, mas hoje não há retry, nem fila.

Para evoluir, a API criaria uma tarefa por finalidade (fila-email, fila-estoque, fila-analytics), como o Cloud Tasks (GCP) cuidando ativamente de retry, limite de tentativas e Dead Letter Queue (armazenar mensagens não entregues para inspeção manual). Como alternativa na AWS, o serviço mais próximo é o Amazon SQS, assegurando que eventos do mesmo pedido sejam processados em sequência mesmo com múltiplos clientes rodando em paralelo.

Parte 4 – Cloud AWS

A aplicação já roda em uma imagem Docker, que pode ser publicada no Amazon ECR e executada no Amazon ECS com Fargate, atrás de um Load Balancer que distribui o tráfego entre as cópias da API e remove automaticamente qualquer uma indisponível, garantindo escalabilidade e balanceamento.

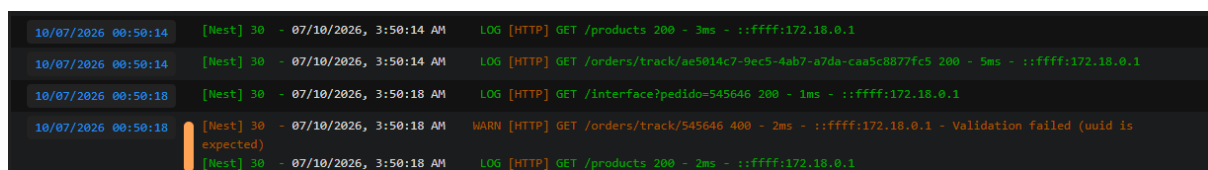
O banco PostgreSQL ficaria no Amazon RDS, que cuida de backups e atualizações automaticamente, e arquivos estáticos/mídias no Amazon

S3. Para monitoramento, o CloudWatch centraliza logs, métricas e alertas. Em segurança, segredos como a chave JWT e a senha do banco ficariam no secrets manager em vez de arquivo .env, e o banco numa rede privada acessível só pela API.

Parte 5 – Observabilidade

Neste projeto foi utilizado o Dozzle junto ao docker, como ferramenta de logs (informação, warning, erro, etc), onde é gerado logs para todos os containers (API, banco de dados e e-mail) e são expostos em uma interface web. Para este teste é uma solução leve e adequada mas que para produção teria que ser alterada (Grafana, Datadog, CloudWatch) pois tem limitações, como salvar o histórico.

Na API foi implementado um interceptor do nest para registrar estes logs a cada requisição. Ainda assim é possível identificar endpoints com gargalos ou erros e criar alertas que podem ser enviados por emails para análises.



```
10/07/2026 00:50:14 [Nest] 30 - 07/10/2026, 3:50:14 AM LOG [HTTP] GET /products 200 - 3ms - ::ffff:172.18.0.1
10/07/2026 00:50:14 [Nest] 30 - 07/10/2026, 3:50:14 AM LOG [HTTP] GET /orders/track/ae5014c7-9ec5-4ab7-a7da-caa5c8877fc5 200 - 5ms - ::ffff:172.18.0.1
10/07/2026 00:50:18 [Nest] 30 - 07/10/2026, 3:50:18 AM LOG [HTTP] GET /interface?pedido=545646 200 - 1ms - ::ffff:172.18.0.1
10/07/2026 00:50:18 [Nest] 30 - 07/10/2026, 3:50:18 AM WARN [HTTP] GET /orders/track/545646 400 - 2ms - ::ffff:172.18.0.1 - Validation failed (uuid is expected)
10/07/2026 00:50:18 [Nest] 30 - 07/10/2026, 3:50:18 AM LOG [HTTP] GET /products 200 - 3ms - ::ffff:172.18.0.1
```

Figura - Logs da API no Dozzle.

Parte 6 – Banco de Dados

O banco de dados foi modelado em PostgreSQL de forma relacional, refletindo o sistema de delivery e seus requisitos.

O relacionamento entre as tabelas segue o padrão de chaves estrangeiras: um cliente possui vários pedidos (1:N), um pedido possui vários itens (1:N) e cada item referência um produto (N:1). Essa normalização evita duplicação de dados e mantém a integridade referencial.

Foram criados índices nas colunas como idx_pedidos_cliente_id (cliente_id) e idx_pedidos_status_id (status_id) usados com mais frequência em filtros e junções (JOIN) para buscar pedidos.

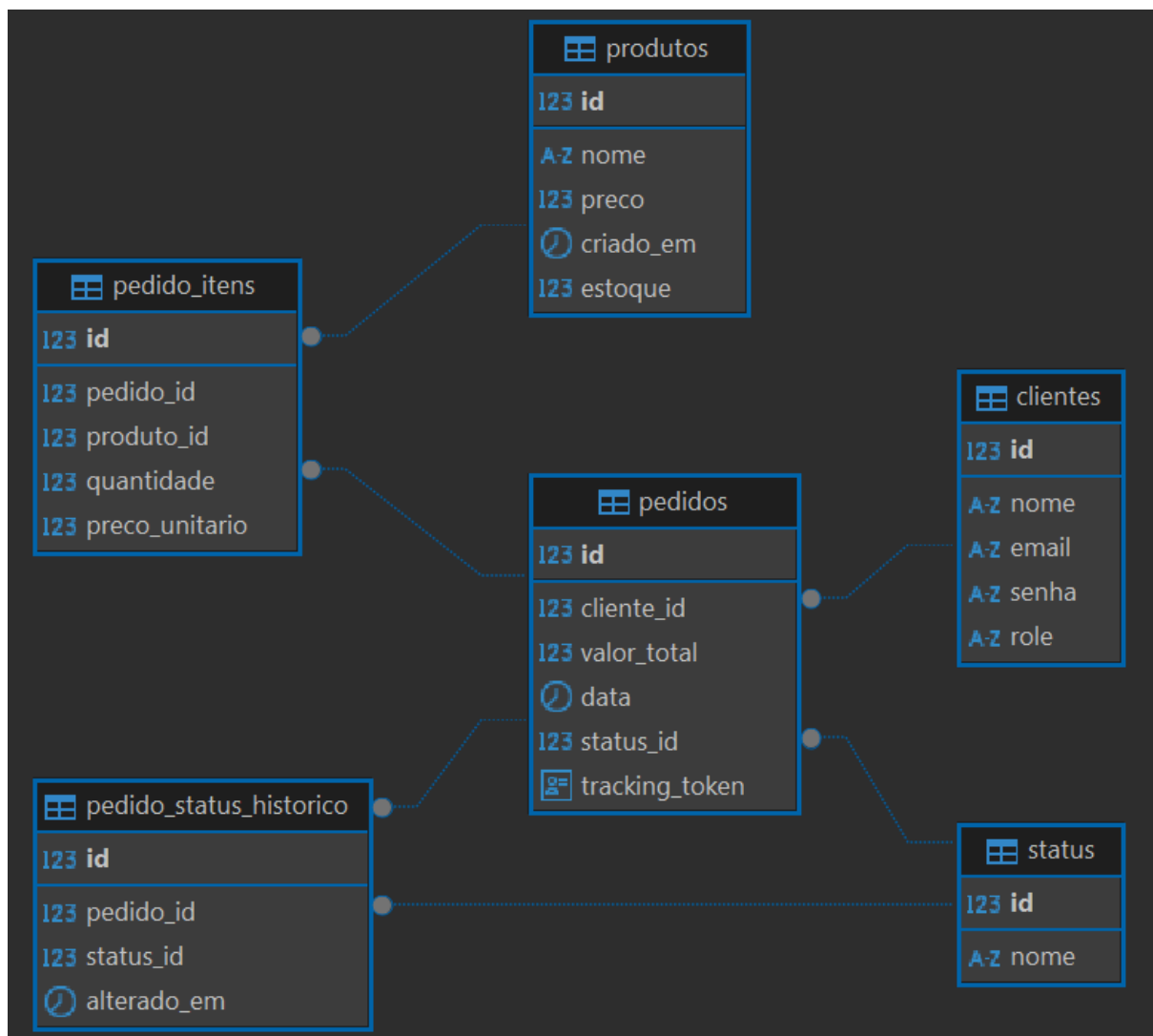


Figura - Diagrama ER do banco de dados.

Parte 7 – Testes

O framework nestjs possui a integração com o jest facilitando a criação de testes definidos nos arquivos .spec para os controllers e serviços dos módulos. No módulo de pedidos, foram implementados testes unitários para criação de pedidos (validando status, produto e estoque), entre outros testes com dados mocados.

Parte 8 – Segurança

Foi implementada autenticação com JWT, no login a senha é validada com *bcrypt* e um *token* assinado com expiração de 1 dia é emitido para ser utilizado nas rotas protegidas.

A autorização usa um RolesGuard, papéis de usuário (ADMIN/CLIENTE), restringindo rotas administrativas e garantindo que um cliente só acesse os próprios pedidos.

Dados sensíveis (JWT secret, credenciais de banco) ficam em variáveis de ambiente fora do controle de versão (.env). TypeORM protege contra SQL Injection via queries parametrizadas, e os Validators do Nest bloqueiam payloads com campos não esperados. Além disso, o CORS (urls do frontend que podem acessar a API) é restrito por variável de ambiente.

Parte 9 – Uso de IA

1. Como você utiliza IA no desenvolvimento de software?

Utilizo principalmente para debater sobre o uso de algum framework para uma funcionalidade específica, comparando diferentes ideias, analisando os pontos positivos e negativos de implementação. Outro ponto importante é automatizar a criação de pull requests com prompts definidos pelo time.

2. Cite um exemplo real onde a IA aumentou sua produtividade.

A IA integrada a projetos auxilia em algumas tarefas que normalmente levariam tempo de codificação, como montar as *models* do banco de dados em uma api por exemplo, tendo o diagrama do banco definido é possível criá-las utilizando IA e assim otimizar o processo de identificação pelo ORM. Outro exemplo importante é gerar a descrição para pull requests, enfatizando diffs e testes.

3. Em quais situações você evita utilizar IA?

Ainda evito usar IA ao implementar serviços que expõem chaves (api keys) sensíveis do projeto, ou em casos em que um serviço pago possa ter carga alta de requisições devido a alguma modificação de IA.

4. Como valida que um código sugerido por IA é seguro e adequado para produção?

Analisando e aprovando cada etapa de ‘pensamento’ da IA, e mesmo após a IA entregar algo, é necessário analisar o fluxo completo manualmente em ambiente de desenvolvimento/homologação.

5. Quais limitações você enxerga no uso de IA para engenharia de software?

Se o prompt estiver equivocado a IA pode implementar algo que não seja o esperado ou que não foi pedido pelo usuário no projeto. Por

isso ainda é necessário a validação e testes do desenvolvedor, tanto do código quanto do prompt.